



Universidade de Santiago de Compostela
Escola Técnica Superior de Enxeñaría
Grao en Enxeñaría Informática

Telaria

Aplicación móbil colaborativa
para a xestión integral de viaxes en grupo

Traballo de Fin de Grao

Jorge Otero Pailos

20 de xuño do 2026

Índice

1. Resumo	3
2. Introducción	4
2.1. Obxectivos xerais	4
2.2. Relación da documentación que conforma a memoria	4
2.3. Descrición do sistema	4
2.4. Información adicional de interese	5
3. Especificación de Requisitos	7
3.1. Descrición xeral	7
3.1.1. Perspectiva do produto	7
3.1.2. Funcionalidade do produto	7
3.1.3. Características dos usuarios	7
3.1.4. Restricións	7
3.1.5. Suposicións e dependencias	7
3.2. Interfaces con outros sistemas	8
3.2.1. Interface de usuario	8
3.2.2. Interfaces de hardware	8
3.2.3. Interfaces de software	8
3.2.4. Interfaces de comunicación	8
3.3. Requisitos funcionais	8
3.4. Casos de uso	9
3.5. Requisitos non funcionais	12
3.5.1. Seguridade	12
3.5.2. Privacidade e protección de datos	12
3.5.3. Rendemento	12
3.5.4. Usabilidade	12
3.5.5. Mantibilidade	12
3.5.6. Portabilidade e despregamento	12
3.5.7. Interoperabilidade	12
3.5.8. Fiabilidade	13
3.6. Información que o sistema debe almacenar	13
4. Deseño	14
4.1. Arquitectura xeral do sistema	14
4.2. Arquitectura do backend	14
4.3. Modelo de datos	15
4.4. Deseño da API REST	17
4.5. Arquitectura do frontend	17
4.6. Fluxos de comunicación entre compoñentes	18
4.6.1. Autenticación e renovación de tokens	18
4.6.2. Chat de grupo en tempo real (SSE)	19
4.6.3. Subida de documentos con URL prefirmada	20
4.6.4. Asistente de IA en <i>streaming</i>	20
4.7. Seguridade	21
4.8. Integración con servizos externos	21
4.9. Equipamento hardware e software	21
5. Probas	22
5.1. Estratexia de probas	22
5.2. Probas automatizadas do backend	22
5.3. Validación funcional do sistema	23
5.4. Resultados acadados	23

6. Conclusións e Posibles Ampliacións	24
6.1. Grao de cumprimento dos obxectivos	24
6.2. Valoración técnica	24
6.3. Posibles ampliacións	24
Bibliografía	26
A. Manual técnico	27
A.1. Requisitos do entorno	27
A.2. Estrutura do repositorio	27
A.3. Servizos de apoio (Podman)	27
A.4. Execución do backend	27
A.5. Execución do frontend	27
A.6. Contrato OpenAPI e xeración de código	28
A.7. Execución das probas	28
A.8. Configuración e segredos	28
A.9. Integración continua	28
B. Manual de usuario	29
B.1. Instalación e acceso	29
B.2. Rexistro e inicio de sesión	29
B.3. Pantalla principal: as miñas viaxes	29
B.4. Crear e unirse a viaxes	29
B.5. Xestión de membros	30
B.6. Gastos e liquidacións	30
B.7. Eventos e calendario	31
B.8. Documentos compartidos	31
B.9. Chat de grupo	31
B.10. Asistente de intelixencia artificial	32
B.11. Amizades	32
B.12. Perfil e axustes	32
B.13. Eliminación da conta	33
B.14. Mensaxes de erro comúns	33

1. Resumo

Este Traballo de Fin de Grao ten como obxectivo o deseño e desenvolvemento de **Telaria**, unha aplicación móbil colaborativa para a xestión integral de viaxes en grupo. A aplicación permite coordinar de forma centralizada todos os aspectos dunha viaxe compartida: o seguimento de gastos e débedas entre os participantes, a planificación de eventos mediante un calendario común, o almacenamento de documentos relevantes, a comunicación entre os membros mediante un chat de grupo e a asistencia dun chatbot conversacional baseado en intelixencia artificial.

A implementación levouse a cabo empregando tecnoloxías modernas para o desenvolvemento de aplicacións móbiles multiplataforma, concretamente **React Native** con **Expo** no frontend e **Spring Boot** no backend, seguindo unha arquitectura *API-first* baseada na especificación **OpenAPI**. A infraestrutura inclúe ademais servizos de almacenamento de ficheiros, modelos de linguaxe locais e despregamento mediante contedores.

Como conclusión, o proxecto ofrece unha solución completa ao problema de coordinación habitual nas viaxes en grupo, integrando nunha única plataforma funcionalidades que adoitan xestionarse de forma dispersa entre distintas ferramentas.

2. Introducción

2.1. Obxectivos xerais

O obxectivo principal deste traballo é o deseño e desenvolvemento de **Telaria**, unha aplicación móbil colaborativa para a xestión integral de viaxes en grupo. O problema que motiva o proxecto é a dispersión habitual das ferramentas empregadas durante a planificación e o transcurso dunha viaxe compartida: os gastos xestiónanse nun grupo de mensaxería, os eventos en aplicacións de calendario individuais, os documentos en servizos de almacenamento xenéricos e a comunicación en plataformas non específicas para o contexto da viaxe.

Telaria centraliza nun único sistema os seguintes aspectos:

- Rexistro de gastos compartidos e cálculo automático das liquidacións entre membros, minimizando o número de transferencias necesarias para saldar as débedas.
- Planificación de eventos mediante un calendario común, con soporte para localización xeográfica.
- Almacenamento de documentos relevantes para a viaxe (reservas, billetes, etc.).
- Chat de grupo entre os membros da viaxe.
- Asistente conversacional baseado en intelixencia artificial, con historial persistente por viaxe.

Á marxe destes obxectivos centrais, a aplicación incorpora unha funcionalidade complementaria de carácter social —unha rede de amizades entre usuarios— que non constitúe un módulo principal, senón unha mellora da experiencia de uso. O seu propósito é simplificar as invitacións recorrentes entre usuarios habituais e reforzar a retención, facilitando que as persoas que proban a aplicación sigan empregándoa en sucesivas viaxes.

Como obxectivos técnicos, o proxecto busca aplicar unha arquitectura *API-first* e tecnoloxías multiplataforma modernas, de xeito que a solución sexa funcional tanto en iOS como en Android desde unha única base de código.

2.2. Relación da documentación que conforma a memoria

Sección	Contido
Resumo	Breve resumo das principais contribucións do traballo.
Introdución	Obxectivos xerais, descrición do sistema e tecnoloxías e arquitecturas empregadas.
Especificación de Requisitos	Requisitos funcionais e non funcionais, interfaces con outros sistemas e información que o sistema debe almacenar.
Deseño	Arquitectura do sistema, división en compoñentes e comunicación entre eles. Equipamento hardware e software empregado.
Probas	Plan de probas con evidencias que verifican a funcionalidade e corrección global do sistema. Resultados acadados.
Conclusións e Posibles Ampliacións	Grao de cumprimento dos obxectivos e posibles vías de mellora futura.
Bibliografía	Lista completa de referencias citadas no texto.
Manuais Técnicos	Documentación para desenvolvedores: código fonte, recursos necesarios e operacións de modificación e proba.
Manuais de Usuario	Documentación autocontida para usuarios finais: instalación, uso, configuración e mensaxes de erro.

2.3. Descrición do sistema

Telaria é unha aplicación móbil orientada a grupos de persoas que realizan viaxes conxuntas. Calquera usuario pode rexistrarse na plataforma e, a partir de aí, crear novas viaxes ou unirse a viaxes existentes mediante invitación, solicitude ou código QR.

Dentro de cada viaxe, todos os membros teñen acceso completo e en igualdade de condicións ás funcionalidades da aplicación. Non existe distinción de roles: o sistema está deseñado baixo un modelo plenamente colaborativo no que calquera participante pode rexistrar gastos, crear eventos, subir documentos, enviar mensaxes ao chat e interactuar co asistente de IA.

O módulo de gastos calcula automaticamente as liquidacións entre membros aplicando un algoritmo de minimización de débedas, de xeito que o número de transferencias necesarias para saldar todos os saldos sexa o mínimo posible. O módulo de eventos permite asociar localizacións xeográficas a cada entrada do calendario. O módulo de documentos delega o almacenamento nun servizo de obxectos compatible con S3 e xera de forma asíncrona miniaturas de previsualización para os documentos de imaxe. O asistente de IA mantén un historial de conversación independente por viaxe, de modo que o contexto se preserve entre sesións.

De maneira complementaria, a plataforma ofrece unha rede de amizades entre usuarios: calquera usuario pode engadir outros como amigos e, a partir de aí, convidalos ás súas viaxes de forma máis áxil que mediante o identificador ou o código QR. Esta funcionalidade non altera o modelo colaborativo descrito, senón que actúa como atallo para os usuarios habituais. Adicionalmente, cada usuario pode eliminar a súa propia conta, operación que require a confirmación do contrasinal.

2.4. Información adicional de interese

Arquitectura API-first con OpenAPI

O deseño do sistema parte da especificación da API antes de calquera implementación. Isto establece un contrato explícito e verificable entre frontend e backend, permite xerar documentación interactiva de forma automática (Swagger UI) [6] e facilita o desenvolvemento en paralelo de ambos os extremos.

React Native e Expo

O frontend está desenvolvido con React Native [1], o que permite compartir a mesma base de código para iOS e Android. Expo [2] engade sobre React Native unha capa de utilidades que simplifica a xestión de dependencias nativas, o proceso de compilación e a distribución da aplicación, reducindo a complexidade de configuración do proxecto.

Spring Boot

O backend emprega Spring Boot [4] pola súa madurez e polo seu ecosistema integrado: Spring MVC para a capa REST, Spring Data JPA para a persistencia, Spring Security para a autenticación e autorización, e integración directa con bibliotecas de xeración de documentación OpenAPI.

Autenticación baseada en JWT

O mecanismo de autenticación utiliza dous tokens complementarios. O *access token*, de tipo JWT [8] e vida curta, inclúese en cada petición á API e permite verificar a identidade do usuario sen consultar a base de datos. O *refresh token*, de tipo opaco e almacenado no servidor, empregase exclusivamente para renovar o *access token* e pode invalidarse explicitamente no peche de sesión.

Ollama

O asistente conversacional funciona sobre Ollama [10], un motor de inferencia local de modelos de linguaxe grande. A execución local elimina a dependencia de servizos externos de pago, preserva a privacidade dos datos dos usuarios e ofrece latencias predecibles independentes de terceiros.

Almacenamento de obxectos compatible con S3

Os documentos compartidos almacénanse nun servizo compatible co protocolo S3 [11]. A elección deste estándar garante a independencia do provedor e aproveita o soporte nativo dispoñible en Spring Boot mediante o AWS SDK.

PostgreSQL

A base de datos relacional escollida é PostgreSQL [7], pola súa robustez, polo seu soporte nativo para UUID e tipos de data con fuso horario, e pola súa excelente integración con Spring Data JPA.

Podman

O despregamento dos servizos realízase mediante contedores OCI xestionados con Podman [13]. A diferenza de Docker, Podman opera sen un daemon centralizado e permite executar os contedores sen privilexios de superusuario (*rootless*), o que mellora o illamento de seguridade do entorno de execución.

Tarefas periódicas de limpeza

Para os documentos de imaxe xérase unha miniatura de previsualización reducida que se mostra na listaxe de documentos sen necesidade de descargar o ficheiro orixinal. Esta xeración realízase de forma asíncrona (fóra do fluxo de subida e da ruta de petición) mediante unha tarefa programada que procesa periodicamente os documentos aínda sen miniatura. Deste xeito a subida e a listaxe de documentos manteñen unha latencia baixa e independente do custo de procesamento de imaxe.

3. Especificación de Requisitos

A especificación de requisitos segue a estrutura do estándar **IEEE 830** para documentos de especificación de requisitos software (SRS), adaptada ás necesidades deste proxecto: unha descrición xeral do produto, as interfaces con outros sistemas, os requisitos funcionais e non funcionais, os casos de uso máis relevantes e a información que o sistema debe almacenar.

3.1. Descrición xeral

3.1.1. Perspectiva do produto

Telaria é unha aplicación móbil baseada nunha arquitectura cliente-servidor. O frontend, desenvolvido con React Native e Expo, comunícase cun backend Spring Boot mediante unha API REST documentada con OpenAPI. O backend integra á súa vez servizos externos: un motor de inferencia de linguaxe (Ollama) para o asistente de IA e un almacén de obxectos compatible con S3 para os documentos compartidos.

3.1.2. Funcionalidade do produto

O sistema cobre os seguintes módulos funcionais:

- Autenticación e xestión de conta de usuario
- Creación e adhesión a viaxes, xestión de membros
- Rexistro de gastos compartidos e cálculo de liquidacións
- Planificación de eventos con localización xeográfica
- Almacenamento de documentos compartidos
- Chat de grupo entre membros da viaxe
- Asistente conversacional de intelixencia artificial por viaxe

Ademais dos módulos anteriores, o sistema inclúe dúas funcionalidades complementarias: unha rede de amizades entre usuarios (funcionalidade de calidade de vida orientada a axilizar as invitacións recorrentes) e a xestión da propia conta, incluída a súa eliminación con confirmación de contrasinal.

3.1.3. Características dos usuarios

Tipo	Descrición
Usuario	Persoa rexistrada na aplicación. Pode crear ou unirse a viaxes e, dentro delas, ten acceso completo a todas as funcionalidades de forma colaborativa e en igualdade de condicións co resto de membros.

3.1.4. Restricións

- A aplicación é multiplataforma (iOS e Android) e funciona sobre Expo.
- Toda comunicación co servidor realízase mediante HTTP e API REST.
- O sistema require conexión á rede para a totalidade das súas operacións.

3.1.5. Suposicións e dependencias

- O servizo Ollama está dispoñible e accesible no servidor para o módulo de IA.
- O servizo de almacenamento PostgreSQL está operativo e accesible para a persistencia de datos.
- Existe un servizo de almacenamento compatible co protocolo S3 para os documentos.
- Os requisitos aquí descritos son estables durante o desenvolvemento.

3.2. Interfaces con outros sistemas

3.2.1. Interface de usuario

Interface gráfica móbil nativa (React Native) con navegación en pestanas e pilas de pantallas, adaptada a iOS e Android sen necesidade de compilación separada por plataforma.

3.2.2. Interfaces de hardware

- Dispositivo móbil do usuario con sistema operativo iOS ou Android e conexión á rede.
- Servidor con capacidade para executar contedores Podman.

3.2.3. Interfaces de software

Sistema	Descrición
API REST propia	Interface principal entre frontend e backend; documentada con OpenAPI/Swagger e consumida mediante JSON sobre HTTP.
Ollama	Servizo local de inferencia de modelos de linguaxe grande, accedido por HTTP para alimentar o asistente conversacional.
Almacenamento S3	Servizo de almacenamento de obxectos (compatible con S3) para a subida e descarga de documentos compartidos.
PostgreSQL	Base de datos relacional para a persistencia de todas as entidades do sistema.

3.2.4. Interfaces de comunicación

O frontend e o backend comunícanse mediante HTTP. A autenticación baséase en dous tokens: un *access token* JWT de vida curta incluído en cada petición, e un *refresh token* opaco de vida longa almacenado no servidor e utilizado para renovar o *access token* sen necesidade de volver a autenticarse.

3.3. Requisitos funcionais

ID	Nome	Descrición	Prior.
RF01	Rexistro de usuario	O sistema permite crear unha conta con nome de usuario, correo electrónico e contrasinal.	Alta
RF02	Autenticación	O sistema autentica usuarios mediante un <i>access token</i> JWT; o <i>refresh token</i> opaco invalidase no peche de sesión.	Alta
RF03	Crear viaxe	Un usuario autenticado pode crear novas viaxes.	Alta
RF04	Unirse a viaxe	Un usuario pode unirse a unha viaxe mediante invitación recibida, solicitude de unión ou código QR.	Alta
RF05	Xestión de membros	Calquera membro pode invitar outros usuarios á viaxe e xestionar as solicitudes de unión pendentes.	Alta
RF06	Rexistro de gastos	Calquera membro pode rexistrar un gasto con importe, nome, categoría, usuario pagador e conxunto de beneficiarios.	Alta
RF07	Cálculo de liquidacións	O sistema calcula automaticamente as liquidacións entre membros, minimizando o número de transferencias necesarias para saldar as débedas.	Alta
RF08	Xestión de eventos	Calquera membro pode crear, consultar e eliminar eventos con nome, data e hora de inicio, duración e localización xeográfica (nome, enderezo, coordenadas e URL de mapa).	Alta

ID	Nome	Descrición	Prior.
RF09	Documentos compartidos	Calquera membro pode subir, consultar e eliminar documentos asociados á viaxe; os documentos de imaxe mostran unha miniatura de previsualización xerada automaticamente.	Media
RF10	Chat de grupo	Os membros dunha viaxe poden enviar e consultar mensaxes de texto no chat común da viaxe.	Media
RF11	Asistente de IA	Cada viaxe dispón dun chatbot conversacional con historial persistente entre sesións.	Baixa
RF12	Rede de amizades	Funcionalidade complementaria: un usuario pode enviar solicitudes de amizade a outro (polo seu identificador ou código QR), aceptalas ou rexeitalas, consultar a súa lista de amigos e eliminar amizades.	Baixa
RF13	Convidar amigos á viaxe	Funcionalidade complementaria que permite convidar amigos a unha viaxe directamente desde a lista de amizades, como atallo ás invitacións por identificador ou QR.	Baixa
RF14	Eliminación de conta	Un usuario pode eliminar a súa propia conta; a operación require a confirmación do contrasinal e elimina os datos persoais asociados.	Baixa
RF15	Xestión do perfil	Un usuario pode consultar e actualizar o seu nome de usuario e correo electrónico (o cambio de correo require a confirmación do contrasinal), cambiar o contrasinal e subir ou consultar o seu avatar.	Media

3.4. Casos de uso

A partir dos requisitos funcionais identifícanse os casos de uso máis relevantes do sistema. Dado que non existe distinción de roles, o único actor é o **Usuario** rexistrado, que interactúa con todas as funcionalidades. A Figura 1 resume os casos de uso agrupados por área funcional, e a continuación detállanse os fluxos principais.

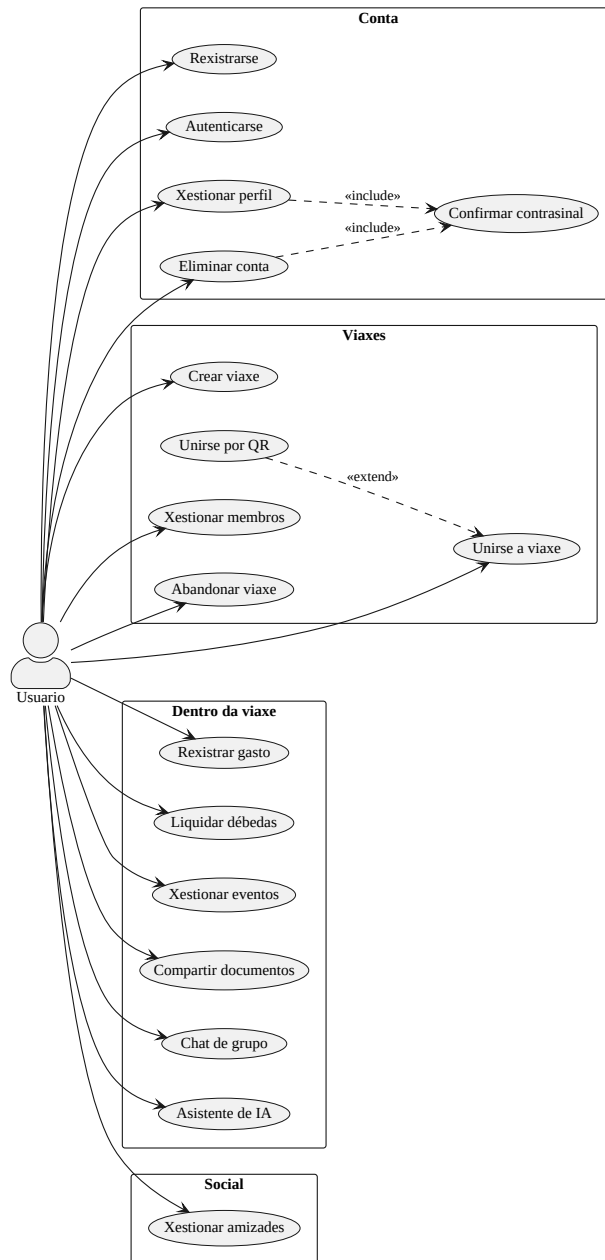


Figura 1: Diagrama de casos de uso de Telaria (actor único: Usuario).

CU01 — Rexistro e autenticación.

- **Actor:** Usuario.
- **Precondicións:** dispón da aplicación instalada e de conexión á rede.
- **Fluxo principal:** (1) introduce nome de usuario, correo e contrasinal e solicita o rexistro; (2) o sistema crea a conta, almacena o contrasinal cifrado e devolve un *access token* JWT e un *refresh token*; (3) o usuario queda autenticado. O inicio de sesión posterior realízase con correo e contrasinal.
- **Fluxos alternativos:** correo xa rexistrado → o sistema rexeita o rexistro; credenciais incorrectas no inicio de sesión → erro de autenticación; caducidade do *access token* → o cliente renóvao de forma transparente co *refresh token*.
- **Poscondición:** o usuario dispón dunha sesión activa.
- **Requisitos:** RF01, RF02.

CU02 — Unirse a unha viaxe.

- **Actor:** Usuario.
- **Precondicións:** usuario autenticado.
- **Fluxo principal:** (1) obtén unha invitación, coñece o identificador ou escanea o código QR dunha viaxe; (2) o sistema rexistra a operación correspondente; (3) o usuario pasa a ser membro da viaxe.
- **Fluxos alternativos:** por invitación (un membro convidado e o usuario acéptaa); por solicitude (o usuario solicita unirse e calquera membro a aproba); por QR (o escaneo resólvese nunha das vías anteriores).
- **Poscondición:** o usuario figura na lista de membros da viaxe.
- **Requisitos:** RF04, RF05.

CU03 — Rexistrar un gasto e liquidar débedas.

- **Actor:** Usuario (membro da viaxe).
- **Precondicións:** o usuario é membro da viaxe.
- **Fluxo principal:** (1) rexistra un gasto (importe, nome, categoría, pagador e beneficiarios); (2) o sistema garda o gasto e recalcula os balances; (3) o membro consulta quen debe a quen; (4) cando se salda unha débeda, rexístrase a liquidación correspondente.
- **Fluxos alternativos:** o pagador ou algún beneficiario non é membro da viaxe → erro de validación.
- **Poscondición:** os balances reflicten o novo gasto; as liquidacións rexistradas reducen as débedas pendentes.
- **Requisitos:** RF06, RF07.

CU04 — Xestionar os membros da viaxe.

- **Actor:** Usuario (membro da viaxe).
- **Precondicións:** o usuario é membro da viaxe.
- **Fluxo principal:** (1) convida a outro usuario (por identificador, amizade ou QR) ou consulta as solicitudes de unión pendentes; (2) acepta ou rexeita unha solicitude; (3) o sistema actualiza a lista de membros.
- **Fluxos alternativos:** usuario xa membro ou xa convidado → conflito.
- **Poscondición:** a composición da viaxe queda actualizada. Calquera membro pode realizar estas accións: non hai roles.
- **Requisitos:** RF05, RF13.

CU05 — Subir un documento.

- **Actor:** Usuario (membro da viaxe).
- **Precondicións:** o usuario é membro da viaxe.
- **Fluxo principal:** (1) inicia a subida indicando nome e tipo de contido; (2) o sistema crea o rexistro e devolve unha URL prefirmada; (3) o cliente sobe o ficheiro directamente ao almacén de obxectos; (4) o cliente confirma a subida e o sistema marca o documento como dispoñible, xerando unha miniatura de forma asíncrona se é unha imaxe.
- **Fluxos alternativos:** se a subida non se confirma, unha tarefa programada elimina o rexistro orfo.
- **Poscondición:** o documento queda listado e descargable mediante URL prefirmada.
- **Requisitos:** RF09.

CU06 — Eliminar a conta.

- **Actor:** Usuario.
- **Precondicións:** usuario autenticado.
- **Fluxo principal:** (1) solicita eliminar a súa conta; (2) o sistema solicita a confirmación do contrasinal; (3) tras a verificación, elimínanse os datos persoais e os artefactos asociados (avatars, documentos) e retírase o usuario das viaxes compartidas; (4) péchase a sesión.
- **Fluxos alternativos:** contrasinal incorrecto → cancelase a operación.
- **Poscondición:** a conta e os datos persoais quedan eliminados.

- **Requisitos:** RF14, RF15.

3.5. Requisitos non funcionais

3.5.1. Seguridade

- **RNF-SE-01** — Control de acceso: só os membros dunha viaxe acceden aos seus datos; calquera usuario autenticado pode crear viaxes.
- **RNF-SE-02** — Os contrasinais almacénanse con hash BCrypt; os *access tokens* JWT teñen vida curta e os *refresh tokens* opacos almacénanse no servidor para limitar a exposición ante compromisos.
- **RNF-SE-03** — As operacións sensíbles sobre a conta —o cambio de correo electrónico e a eliminación da conta— requiren a confirmación do contrasinal actual.

3.5.2. Privacidade e protección de datos

- **RNF-PR-01** — A eliminación da conta borra os datos persoais do usuario e os artefactos asociados no almacén de obxectos (avatar e documentos), e retírao das viaxes compartidas.
- **RNF-PR-02** — As mensaxes de chat (de grupo e do asistente de IA) elimínanse automaticamente tras un período de retención de 30 días mediante unha tarefa programada.

3.5.3. Rendemento

- **RNF-RE-01** — O cálculo de liquidacións realízase no servidor, evitando sobrecargar o dispositivo cliente.
- **RNF-RE-02** — A entrega de mensaxes do chat e das respostas do asistente de IA realízase en tempo real mediante eventos enviados polo servidor (SSE).
- **RNF-RE-03** — A xeración de miniaturas dos documentos de imaxe execútase de forma asíncrona, fóra da ruta de petición, para non penalizar a latencia da subida nin da listaxe.

3.5.4. Usabilidade

- **RNF-US-01** — A interface debe ser intuitiva e usable sen formación previa.
- **RNF-US-02** — A aplicación estará dispoñible en galego, castelán e inglés.

3.5.5. Mantibilidade

- **RNF-MA-01** — O contrato OpenAPI é a fonte de verdade da API: a partir del xéranse automaticamente as interfaces e DTO do backend e os tipos do frontend, garantindo a sincronía entre ambos os extremos.
- **RNF-MA-02** — A lóxica do backend está cuberta por unha batería de probas automatizadas (unitarias e de integración) que facilita a evolución segura do código.

3.5.6. Portabilidade e despregamento

- **RNF-PO-01** — O backend e os servizos auxiliares despreganse en contedores Podman.
- **RNF-PO-02** — A aplicación móbil é compatible con iOS e Android mediante Expo/React Native.

3.5.7. Interoperabilidade

- **RNF-IN-01** — O almacenamento de documentos baséase no protocolo estándar S3, o que garante a independencia respecto dun provedor concreto.

3.5.8. Fiabilidade

- **RNF-FI-01** — O sistema valida os datos de entrada na API e devolve mensaxes de erro claras e estruturadas ao cliente.

3.6. Información que o sistema debe almacenar

Entidade	Atributos principais
Usuario	Identificador (UUID), nome de usuario, correo electrónico, contrasinal (hash).
Viaxe	Identificador (UUID), nome, conxunto de membros (usuarios).
Evento	Identificador, nome, data e hora de inicio (con fuso horario), duración en minutos, localización embebida (nome, enderezo, latitude, lonxitude, URL de mapa).
Gasto	Identificador, importe (decimal), nome, categoría (enumeración), usuario pagador, usuario creador do rexistro, conxunto de beneficiarios, marca temporal.
Liquidación	Identificador, importe (decimal), usuario pagador, usuario receptor, marca temporal.
Documento compartido	Identificador, nome de ficheiro, clave no almacén de obxectos, clave da miniatura de previsualización, estado de subida (booleano), usuario creador, marca temporal.
Invitación / Solicitud	Identificador, usuario, viaxe asociada, marca temporal de creación.
Amizade	Identificador (UUID), dous usuarios asociados (almacenados de forma normalizada para garantir a unicidade da parella), marca temporal de creación.
Solicitud de amizade	Identificador (UUID), usuario emisor, usuario receptor, marca temporal de creación.
Mensaxe de chat de grupo	Identificador, contido (texto), autor, viaxe asociada, marca temporal.
Mensaxe de asistente de IA	Identificador, contido (texto), rol (USUARIO ou ASISTENTE), usuario, viaxe asociada, marca temporal.
Token de refresco	Token (identificador), identificador do usuario asociado.

4. Deseño

Neste capítulo descríbese como se constrúe o sistema: a súa arquitectura xeral, a división en compoñentes e a comunicación entre eles, o modelo de datos, o deseño da API e dos extremos cliente e servidor, os fluxos máis relevantes e, por último, o equipamento hardware e software empregado. Sempre que é posible recórrese a representacións gráficas para facilitar a comprensión da estrutura do produto.

4.1. Arquitectura xeral do sistema

Telaria segue unha arquitectura **cliente–servidor** de tres capas lóxicas (presentación, lóxica de negocio e persistencia) e adopta un enfoque *API-first*: o contrato da API, especificado con OpenAPI, é o punto de partida do desenvolvemento e o elemento que vincula o cliente co servidor. A Figura 2 mostra a visión de conxunto.

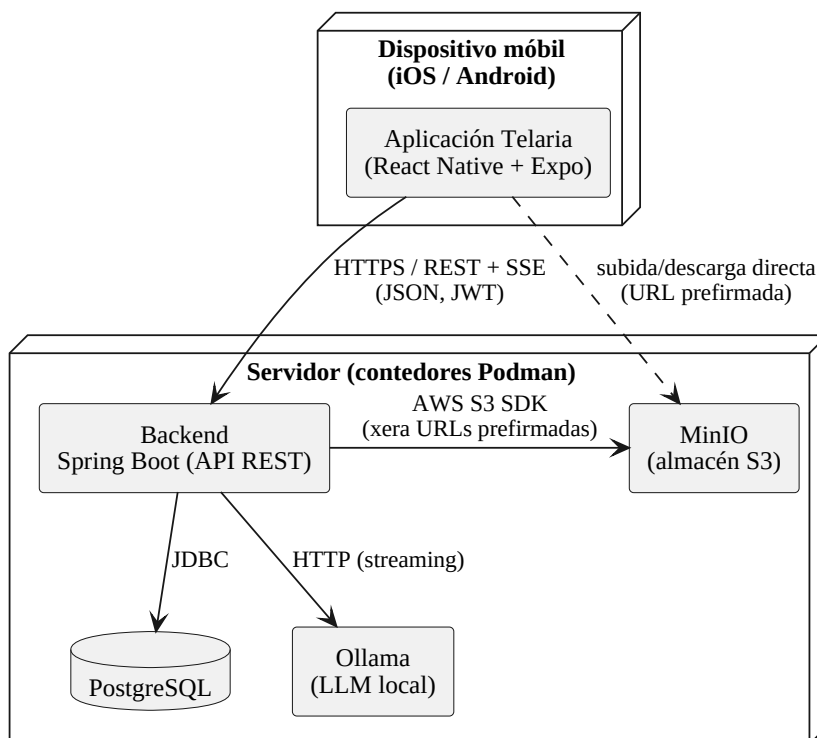


Figura 2: Arquitectura xeral e despregamento de Telaria.

O **cliente** é unha aplicación móbil multiplataforma (React Native + Expo) que se executa en iOS e Android. O **servidor** agrupa, en contedores xestionados con Podman, o backend Spring Boot e os tres servizos de apoio: a base de datos PostgreSQL, o almacén de obxectos compatible con S3 (MinIO) e o motor de inferencia de modelos de linguaxe (Ollama).

A comunicación principal entre cliente e servidor realízase sobre HTTP mediante unha API REST que intercambia JSON e protexe cada petición cun *access token* JWT. Para as funcionalidades en tempo real —chat de grupo e asistente de IA— o servidor emprega *Server-Sent Events* (SSE), que permiten un fluxo unidireccional de eventos do servidor cara ao cliente sobre a mesma conexión HTTP. Finalmente, a subida e a descarga de documentos non pasan polo backend: este limitase a xerar *URLs prefirmadas* co que o cliente fala **directamente** co almacén de obxectos, descargando o servidor da transferencia de ficheiros.

4.2. Arquitectura do backend

O backend organízase nunha arquitectura por capas que separa claramente as responsabilidades, como se aprecia na Figura 3.

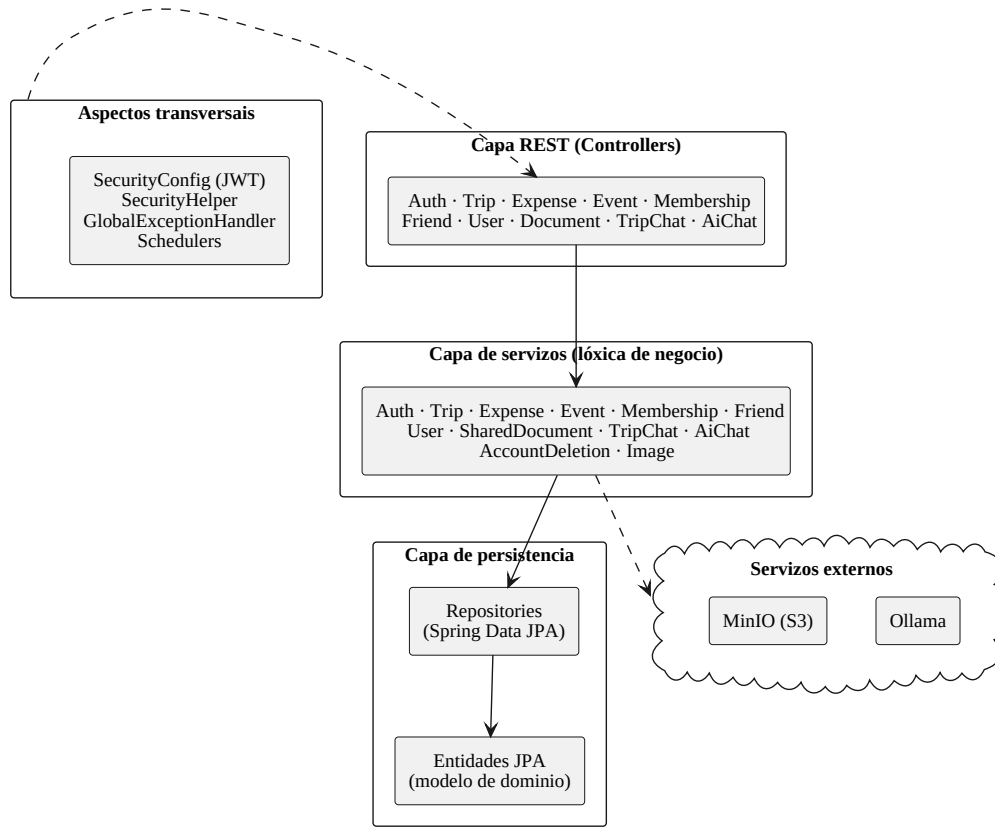


Figura 3: Componentes e capas do backend.

- **Capa REST (controllers).** Expón os *endpoints* da API. As interfaces destes controladores xéranse automaticamente a partir do contrato OpenAPI, e as clases que as implementan conteñen unicamente a tradución entre a petición HTTP e a chamada ao servizo correspondente.
- **Capa de servizos.** Concentra a lóxica de negocio: validacións, cálculo de liquidacións, resolución de invitacións e solicitudes, xestión de ficheiros, etc. É a única capa que coordina varias entidades e repositorios dentro dunha mesma transacción.
- **Capa de persistencia.** Componse dos repositorios de Spring Data JPA e das entidades do modelo de dominio, que Hibernate mapea ás táboas de PostgreSQL.
- **Aspectos transversais.** A configuración de seguridade (validación de tokens JWT con Spring Security), a utilidade de extracción do usuario autenticado (**SecurityHelper**), o manexo centralizado de erros (**GlobalExceptionHandler**, que devolve respostas no formato *Problem Details*, RFC 7807) [9] e as tarefas programadas (*schedulers*) de limpeza e xeración de miniaturas.

A autorización aplícase na capa de servizos: o acceso aos datos dunha viaxe verifícase comprobando que o usuario autenticado pertence aos seus membros, de modo que ningún usuario poida acceder á información de viaxes das que non forma parte.

4.3. Modelo de datos

A Figura 4 representa o modelo de dominio coas entidades persistentes e as súas relacións.

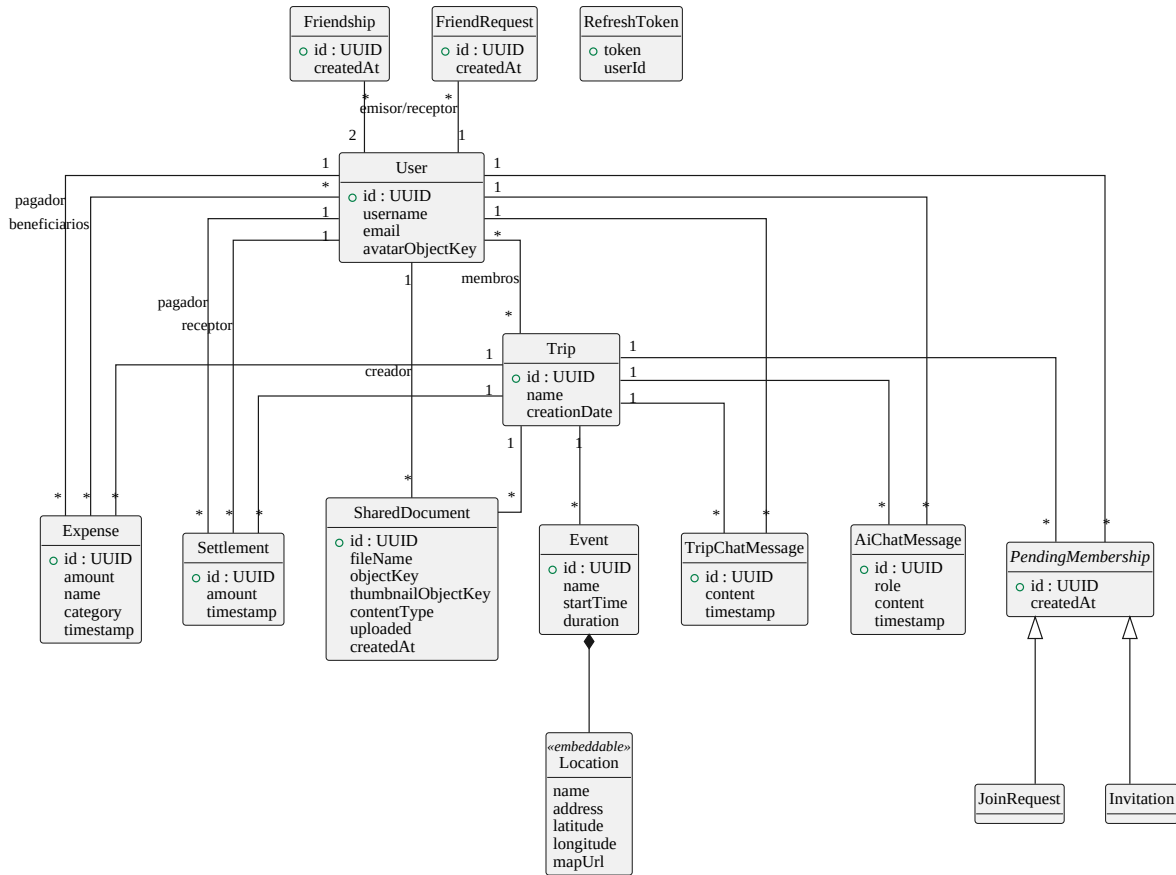


Figura 4: Modelo de datos (entidades e relacións principais).

As decisións de deseño máis salientables son:

- **Identificadores UUID.** Todas as entidades principais empregan UUID como clave primaria, o que evita expor identificadores secuenciais e facilita a xeración de claves no lado da aplicación.
- **Relacións varios-a-varios.** A pertenza ás viaxes (User-Trip) e os beneficiarios dun gasto (Expense-User) modélanse mediante táboas de unión.
- **Normalización das amizades.** Cada **Friendship** almacena os dous usuarios de forma ordenada (o de identificador menor primeiro), o que garante a unicidade da parella e simplifica as consultas bidireccionais.
- **Valor embebido.** A localización dun evento (**Location**: nome, enderezo, coordenadas e URL de mapa) modélase como un obxecto embebido dentro de **Event**, ao non ter identidade propia (considérase que os diferentes eventos non compartirán ubicación frecuentemente).
- **Xerarquía de membros pendentes.** **Invitation** e **JoinRequest** comparten estrutura (viaxe, usuario e marca temporal) a través da superclase **PendingMembership**, diferenciándose só na dirección da petición.
- **Token de refresco persistente.** O **RefreshToken** almacénase no servidor para poder invalidalo no peche de sesión.

O esquema da base de datos non se xestiona con ferramentas de migración: emprégase a xeración automática de Hibernate (**ddl-auto**), apropiada para o contexto deste proxecto. Na sección de conclusións recóllese a adopción de migracións explícitas como posible mellora de cara a un entorno de produción.

4.4. Deseño da API REST

A API é o contrato central do sistema e especificase con **OpenAPI** de forma modular: un documento principal (`api.yaml`) referencia, mediante `$ref`, ficheiros independentes para as rutas (`paths/`) e para os esquemas de datos (`schemas/`). Esta organización mantén o contrato lexible malia o seu tamaño.

A partir deste contrato xérase código automaticamente en ambos os extremos:

- No **backend**, o complemento *OpenAPI Generator* produce as interfaces dos controladores e os obxectos de transferencia de datos (DTO).
- No **frontend**, a ferramenta `openapi-typescript` xera os tipos TypeScript que describen as peticións e respostas.

Deste xeito, calquera cambio no contrato propágase automaticamente como tipos a ambos os extremos, e calquera incoherencia detéctase en tempo de compilación. Ademais, a partir da mesma especificación ofrécese documentación interactiva da API (Swagger UI). A Táboa 3 resume os principais recursos da API.

Recurso	Responsabilidade
<code>/auth</code>	Registro, inicio e peche de sesión, renovación de tokens.
<code>/users</code>	Perfil, contrasinal, avatar, eliminación de conta.
<code>/trips</code>	Creación e consulta de viaxes.
<code>/trips/{tripID}/expenses</code>	Gastos, balances e liquidacións.
<code>/trips/{tripID}/events</code>	Eventos do calendario.
<code>/trips/{tripID}/documents</code>	Documentos compartidos (subida en varios pasos).
<code>/trips/{tripID}/group-chat</code>	Chat de grupo (consulta, envío e fluxo SSE).
<code>/trips/{tripID}/ai-chat</code>	Asistente de IA (historial e resposta en <i>streaming</i>).
Invitacións, solicitudes, amizades	Xestión de membros e da rede social de usuarios.

Cadro 3: Resumo dos principais recursos da API REST.

4.5. Arquitectura do frontend

O cliente organízase tamén por capas, como amosa a Figura 5. A navegación baséase en **Expo Router**, un sistema de rutas baseado na estrutura de ficheiros, que agrupa as pantallas en tres grupos (`(auth)`, `(main)` e `(trip)`).

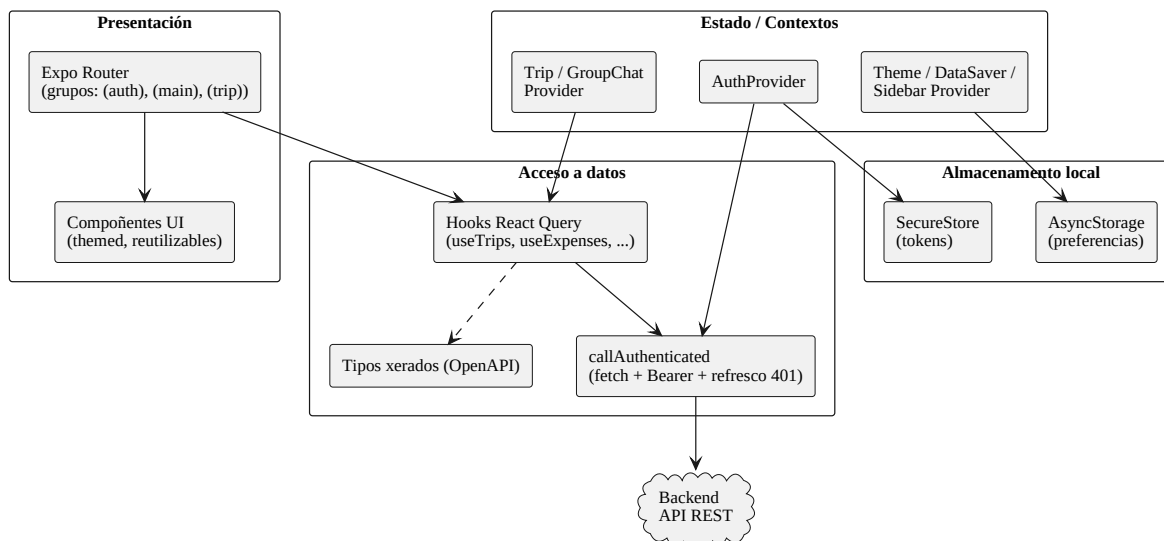


Figura 5: Arquitectura do frontend.

O estado da aplicación divídese en dous tipos. O **estado de servidor** (datos que proveñen do backend) xestiónase con **React Query** [3], que se encarga da caché, da revalidación e da sincronización; cada recurso ten os seus *hooks* de consulta e mutación e unha factoría de claves de caché. O **estado de aplicación** (sesión, tema, viaxe actual, chat) maniféstase mediante contextos de React (**AuthProvider**, **ThemeProvider**, **TripProvider**, **GroupChatProvider**, etc.).

Toda chamada autenticada pasa por unha función envoltorio (**callAuthenticated**) que engade o *access token* á cabeceira **Authorization** e, ante unha resposta 401, renova o token de forma transparente e reintent a petición. Os tokens gárdanse de forma cifrada (**SecureStore**) e as preferencias do usuario en almacenamento local (**AsyncStorage**). A Figura 6 mostra o mapa completo de navegación.

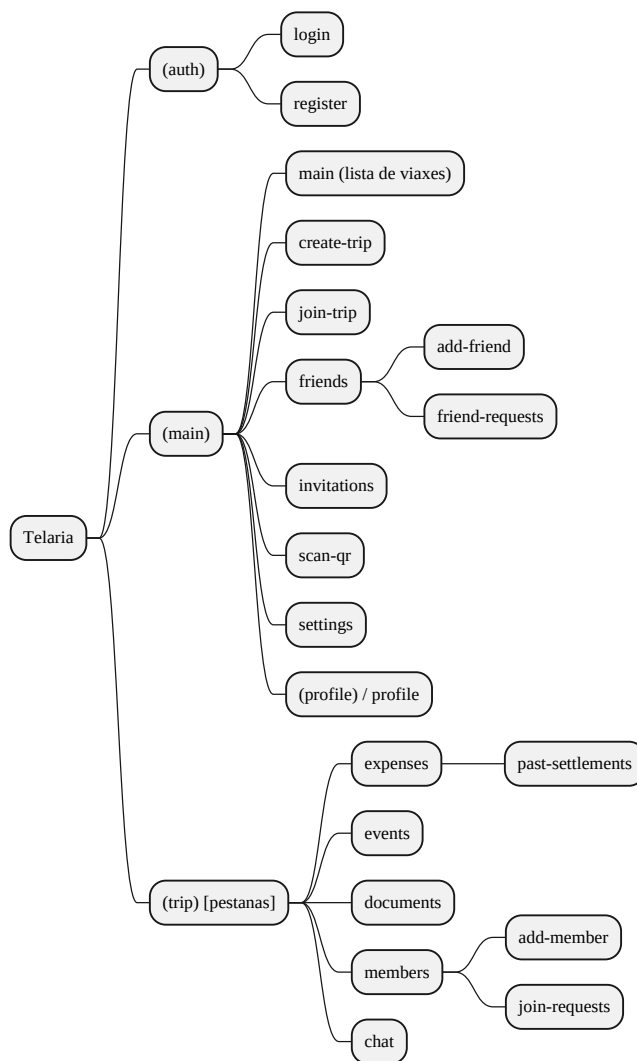


Figura 6: Mapa de navegación da aplicación.

4.6. Fluxos de comunicación entre compoñentes

4.6.1. Autenticación e renovación de tokens

O inicio de sesión devolve un *access token* JWT de vida curta e un *refresh token* opaco. Cando o *access token* caduca, o cliente detéctao ante unha resposta 401 e renóvalo de forma transparente, sen requirir unha nova autenticación do usuario (Figura 7).

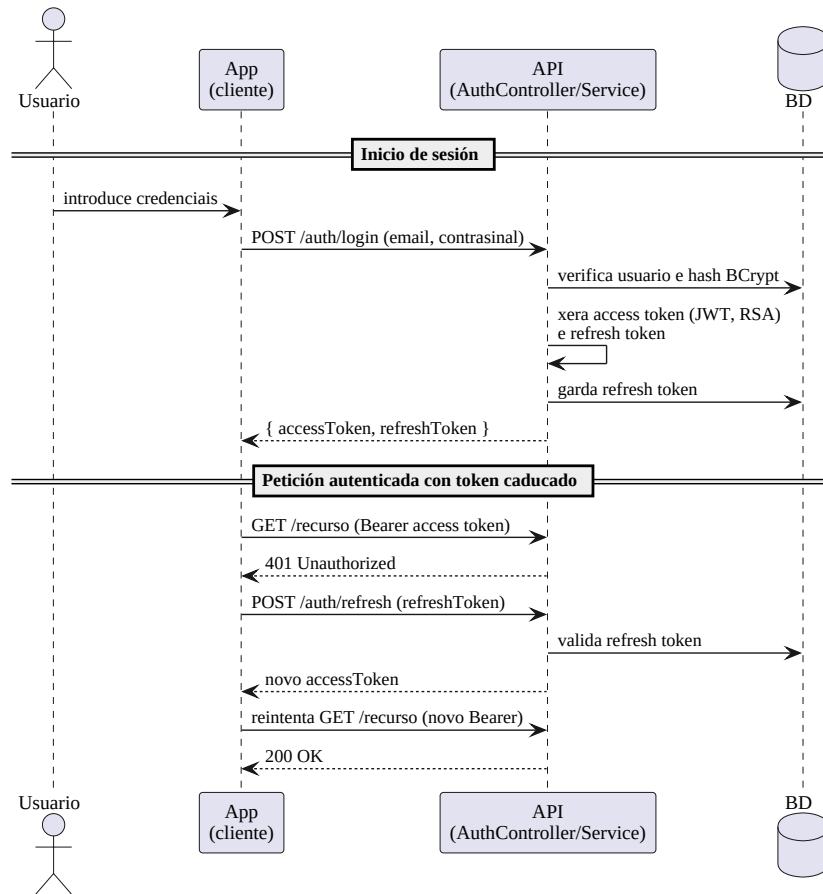


Figura 7: Secuencia de autenticación e renovación do *access token*.

4.6.2. Chat de grupo en tempo real (SSE)

Cando un membro abre o chat, o cliente subscríbese a un fluxo SSE. Ao enviar outro membro unha mensaxe, o servidor persístea e difúndea de inmediato a todos os subscritores conectados (Figura 8).

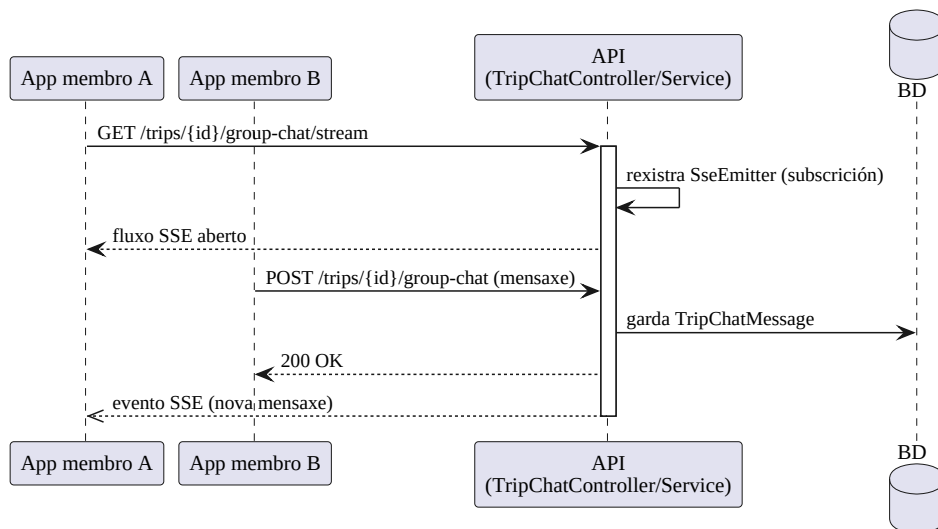


Figura 8: Secuencia do chat de grupo mediante *Server-Sent Events*.

4.6.3. Subida de documentos con URL prefirmada

A subida realízase en varios pasos para que o ficheiro non atravesase o backend: este crea o rexistro e devolve unha URL prefirmada, o cliente sobe o ficheiro directamente ao almacén e, finalmente, confirma a subida; se o ficheiro é unha imaxe, xérase unha miniatura de forma asíncrona (Figura 9).

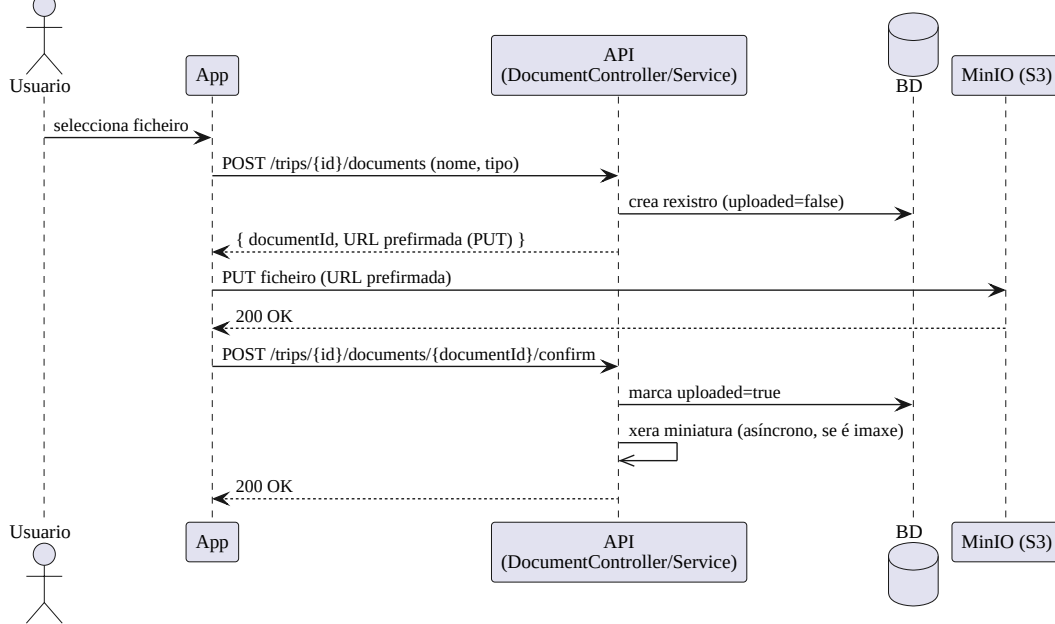


Figura 9: Secuencia de subida de documentos con URL prefirmada.

4.6.4. Asistente de IA en *streaming*

Ao recibir unha consulta, o servidor constrúe o contexto da conversa a partir dos eventos e gastos da viaxe e das últimas mensaxes, e remíttello ao modelo local (Ollama). A resposta devólvese token a token cara ao cliente mediante SSE, ofrecendo unha experiencia de escritura progresiva (Figura 10).

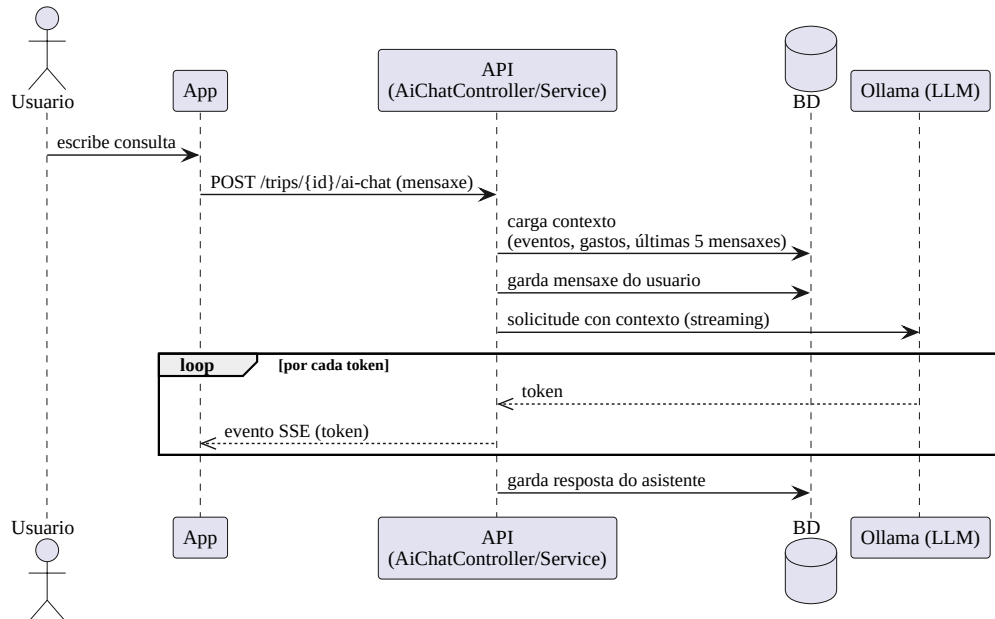


Figura 10: Secuencia do asistente de IA con resposta en *streaming*.

4.7. Seguridade

A seguridade do sistema descansa sobre os seguintes piares de deseño:

- **Autenticación con JWT asinado.** No inicio de sesión, o backend asina os seus propios *access tokens* (JWT) cunha clave privada RSA almacenada nun *keystore*; en cada petición válida a sinatura do token coa clave pública, sen necesidade de consultar a base de datos. Esta validación de tokens portadores (*bearer*) apóiase no soporte de JWT de Spring Security [5]. Non se implementa o protocolo OAuth 2.0: non existe un servidor de autorización externo nin fluxos de delegación, senón un esquema de autenticación propio baseado en tokens autoasinados.
- **Sesións sen estado.** O servidor non mantén sesión; cada petición é autosuficiente grazas ao token, o que mellora a escalabilidade.
- **Contrasinais protexidos.** Os contrasinais almacénanse cifrados con BCrypt [12].
- **Autorización por pertenza.** O acceso aos recursos dunha viaxe está condicionado a que o usuario sexa membro dela.
- **Confirmación en operacións sensibles.** O cambio de correo electrónico e a eliminación de conta requiren reintroducir o contrasinal actual.

4.8. Integración con servizos externos

- **Ollama.** O asistente de IA delega a inferencia nun modelo de linguaxe executado localmente, ao que o backend accede por HTTP en modo *streaming*.
- **Almacén de obxectos (MinIO/S3).** A xestión de ficheiros realízase co AWS SDK; o backend nunca recibe nin serve os ficheiros, senón que xera URLs prefirmadas de subida e descarga cunha validez limitada.
- **Procesamento de imaxes.** As miniaturas de previsualización e o redimensionamento de avatares prodúcense reducindo a imaxe a un tamaño máximo e convertíndoa a JPEG, sempre fóra da ruta de petición.

4.9. Equipamento hardware e software

O desenvolvemento e a execución do sistema requiren o seguinte equipamento, resumido na Táboa 4. Dado que ningunha destas tecnoloxías constituía un requisito previo, a súa elección xustifícase nas seccións anteriores e na introdución.

Ámbito	Tecnoloxías e ferramentas
Frontend	React Native, Expo, TypeScript, React Query; ferramentas de Node.js.
Backend	Java, Spring Boot, Spring Data JPA, Spring Security; construción con Gradle.
Persistencia	PostgreSQL.
Almacenamento	MinIO (compatible con S3).
Intelixencia artificial	Ollama (modelo de linguaxe local).
Despregamento	Contedores OCI xestionados con Podman.
Cliente final	Dispositivo móbil iOS ou Android con conexión á rede.

Cadro 4: Equipamento software e hardware do sistema.

5. Probas

A verificación da funcionalidade e da corrección global do sistema aborda dúas dimensións complementarias: as **probas automatizadas** do backend, que garanten a corrección técnica da lóxica e da API, e a **validación funcional** de extremo a extremo da aplicación, que comproba que o produto se comporta como espera o usuario.

5.1. Estratexia de probas

A estratexia segue o modelo da pirámide de probas, con tres niveis:

- **Probas unitarias:** illan a lóxica de negocio de cada servizo, substituíndo as dependencias por dobres de proba (*mocks*).
- **Probas de integración (E2E):** exercitan a API completa contra unha base de datos e un almacén de obxectos reais, levantados en contedores efémeros.
- **Validación de sistema:** comprobación manual dos fluxos de usuario completos sobre a aplicación en execución.

5.2. Probas automatizadas do backend

O backend conta cunha batería de **367 probas automatizadas** repartidas en 26 clases, que se executan co *framework* JUnit 5. A Táboa 5 resume a súa distribución.

Tipo	Clases	Descrición
Unitarias de servizos	10	Lóxica de negocio illada con Mockito.
Integración (E2E)	11	API completa con base de datos e almacén reais (Testcontainers).
Modelo	3	Lóxica propia das entidades de dominio.
Tarefas programadas	2	Comportamento dos <i>schedulers</i> de limpeza.
Total	26	367 probas

Cadro 5: Distribución das probas automatizadas do backend.

As probas de integración empregan **Testcontainers** [14], que levanta contedores efémeros de PostgreSQL e MinIO para cada execución, de xeito que as probas se realizan contra servizos reais e non contra simulacións. Unha clase base común (**BaseE2ETest**) orquestra o arrinque dos contedores e a aplicación execútase cun perfil de proba que recrea o esquema da base de datos en cada execución. Deste xeito verifícanse os fluxos completos da API: autenticación, xestión de viaxes e membros, gastos e liquidacións, eventos, documentos, chat e asistente de IA.

A cobertura de código mídese con **JaCoCo** [15]. A Táboa 6 resume os resultados obtidos na execución completa da batería.

Métrica	Cobertura
Liñas	55,8 %
Instrucións	48,3 %
Métodos	60,7 %
Clases	85,5 %
Ramas	21,8 %

Cadro 6: Cobertura de código do backend (JaCoCo).

A cobertura concéntrase na capa de servizos e nos fluxos principais da API, que é onde reside a lóxica de negocio. As cifras máis baixas en ramas corresponden, en boa medida, a ramas defensivas e de manexo de erros pouco frecuentes. Ademais, a construción e a execución das probas intégranse nun fluxo de integración continua (GitHub Actions), e o código sométese a análise estática.

5.3. Validación funcional do sistema

Para verificar o comportamento de extremo a extremo executáronse manualmente sobre a aplicación os escenarios de uso descritos na Táboa 7, que cobren a totalidade dos requisitos funcionais. Cada escenario comprende a secuencia completa de accións do usuario e a comprobación do resultado esperado.

ID	RF	Escenario verificado
CP01	RF01, RF02	Rexistro, inicio e peche de sesión, e renovación de token.
CP02	RF03–RF05	Crear viaxe, convidar e aceptar membros (e por código QR).
CP03	RF06, RF07	Rexistrar gastos, consultar balances e liquidar débedas.
CP04	RF08	Crear, consultar e eliminar eventos do calendario.
CP05	RF09	Subir, previsualizar e eliminar documentos.
CP06	RF10	Enviar e recibir mensaxes no chat de grupo en tempo real.
CP07	RF11	Conversar co asistente de IA e recuperar o historial.
CP08	RF12, RF13	Xestionar amizades e convidar amigos a unha viaxe.
CP09	RF14, RF15	Editar o perfil e eliminar a conta con confirmación.

Cadro 7: Escenarios de validación funcional e a súa trazabilidade cos requisitos.

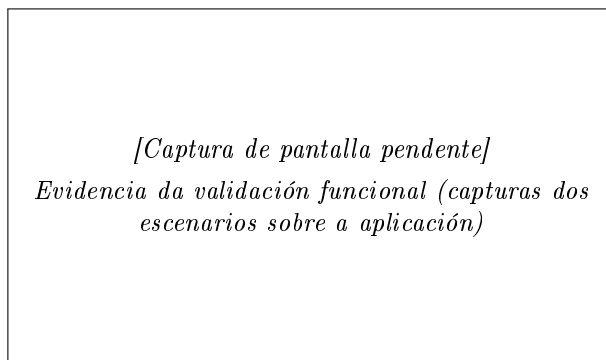


Figura 11: Evidencia da validación funcional (capturas dos escenarios sobre a aplicación)

5.4. Resultados acadados

A totalidade das **367 probas automatizadas pasan correctamente**, e os nove escenarios de validación funcional completáronse co resultado esperado, polo que se verifica o cumprimento de todos os requisitos funcionais. A cobertura de código do backend sitúase arredor do 56 % de liñas, concentrada na lóxica de negocio.

A principal limitación do plan de probas é a ausencia de probas automatizadas no frontend. Este risco mitígasen por dúas vías: a validación funcional manual descrita anteriormente e a barreira de tipos que impón o contrato OpenAPI, que detecta en tempo de compilación as incoherencias entre cliente e servidor. A incorporación de probas automatizadas no frontend recóllese como liña de mellora nas conclusións.

6. Conclusións e Posibles Ampliacións

6.1. Grao de cumprimento dos obxectivos

O proxecto acadou os obxectivos formulados na introdución. Telaria centraliza nunha única aplicación a xestión integral dunha viaxe en grupo, substituíndo o conxunto disperso de ferramentas que se empregan habitualmente. A Táboa 8 resume o grao de cumprimento de cada obxectivo.

Obxectivo	Estado	Observacións
Gastos compartidos e liquidacións	Cumprido	Cálculo no servidor con minimización de transferencias (RF06, RF07).
Calendario de eventos con localización	Cumprido	Eventos con data, duración e localización xeográfica (RF08).
Documentos compartidos	Cumprido	Subida directa ao almacén con miniaturas (RF09).
Chat de grupo	Cumprido	Mensaxería en tempo real mediante SSE (RF10).
Asistente de IA por viaxe	Cumprido	Chatbot con historial persistente sobre Ollama (RF11).
Rede de amizades (complementaria)	Cumprido	Solicitudes, lista de amigos e convite áxil (RF12, RF13).
Xestión de conta	Cumprido	Perfil, contrasinal, avatar e eliminación con confirmación (RF14, RF15).
Arquitectura API-first	Cumprido	Contrato OpenAPI con xeración de código en ambos os extremos.
Multiplataforma iOS/Android	Cumprido	Única base de código con React Native e Expo.

Cadro 8: Grao de cumprimento dos obxectivos.

6.2. Valoración técnica

A adopción dunha arquitectura *API-first* demostrou ser un acerto: ter o contrato OpenAPI como fonte de verdade permitiu xerar automaticamente os tipos do cliente e as interfaces do servidor, mantendo ambos os extremos sincronizados e detectando incoherencias en tempo de compilación. O uso de *Server-Sent Events* para o chat e o asistente de IA, e o das URLs prefirmadas para os ficheiros, permitiron implementar funcionalidades esixentes sen sobrecargar o servidor. A execución local do modelo de linguaxe con Ollama evitou a dependencia de servizos externos de pago. Finalmente, a batería de probas automatizadas do backend proporcionou unha rede de seguridade que facilitou a evolución continua do código.

Entre as dificultades atopadas destacan a integración de Testcontainers cun entorno de contedores sen daemon (Podman), a coordinación dos fluxos asíncronos de *streaming* e a xestión coherente da subida de ficheiros en varios pasos.

6.3. Posibles ampliacións

O sistema deixa aberto un conxunto de liñas de mellora:

- **Notificacións push** para invitacións, mensaxes novas e recordatorios de eventos.
- **Preparación para produción:** externalizar os segredos de configuración (actualmente en ficheiros de configuración), habilitar HTTPS/TLS e facer configurable a dirección do servidor no cliente.
- **Probas automatizadas no frontend** (por exemplo, con Jest e React Native Testing Library), que complementarían a sólida cobertura do backend.
- **Migracións de base de datos explícitas** (Flyway ou Liquibase) en lugar da xeración automática de esquema, de cara a un control de versións do esquema en produción.

- **Visualización de mapas** para a localización dos eventos, aproveitando as coordenadas xa almacenadas.
- **Modo sen conexión** con sincronización posterior, para mellorar a experiencia en destinos con conectividade limitada.

Bibliografia

- [1] Meta Platforms, Inc. *React Native — A framework for building native apps using React*. <https://reactnative.dev>
- [2] Expo. *Expo Documentation*. <https://docs.expo.dev>
- [3] TanStack. *TanStack Query (React Query)*. <https://tanstack.com/query>
- [4] Broadcom (VMware Tanzu). *Spring Boot Reference Documentation*. <https://spring.io/projects/spring-boot>
- [5] Broadcom (VMware Tanzu). *Spring Security Reference Documentation*. <https://spring.io/projects/spring-security>
- [6] OpenAPI Initiative. *OpenAPI Specification*. <https://spec.openapis.org>
- [7] The PostgreSQL Global Development Group. *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
- [8] M. Jones, J. Bradley e N. Sakimura. *JSON Web Token (JWT)*. RFC 7519, IETF, 2015. <https://www.rfc-editor.org/rfc/rfc7519>
- [9] M. Nottingham e E. Wilde. *Problem Details for HTTP APIs*. RFC 7807, IETF, 2016. <https://www.rfc-editor.org/rfc/rfc7807>
- [10] Ollama. *Ollama — Get up and running with large language models locally*. <https://ollama.com>
- [11] MinIO, Inc. *MinIO Object Storage Documentation*. <https://min.io>
- [12] N. Provos e D. Mazières. *A Future-Adaptable Password Scheme*. Proceedings of the USENIX Annual Technical Conference, 1999.
- [13] Red Hat, Inc. *Podman — A tool for managing OCI containers and pods*. <https://podman.io>
- [14] AtomicJar / Docker, Inc. *Testcontainers for Java*. <https://java.testcontainers.org>
- [15] *JaCoCo Java Code Coverage Library*. <https://www.jacoco.org/jacoco/>

A. Manual técnico

Este manual recolle a información necesaria para instalar, executar, probar e modificar Telaria. Diríxese a persoas que vaian continuar o desenvolvemento do sistema.

A.1. Requisitos do entorno

- **JDK 25** (para o backend; a construción realízase co *wrapper* de Gradle).
- **Node.js** con **npm** (para o frontend e a xeración de tipos).
- **Podman** con soporte de `podman compose` (para os servizos de apoio).
- **Git**.

A.2. Estrutura do repositorio

- `backend/` — API REST en Spring Boot (Gradle).
- `frontend/` — aplicación móbil en React Native + Expo.
- `memoria/` — esta documentación (LaTeX).

A.3. Servizos de apoio (Podman)

Os servizos auxiliares (PostgreSQL, MinIO e Ollama) defínense en `backend/compose.yaml`. O xeito máis sinxelo de poñelos en marcha, descargar o modelo de linguaxe e arrincar o backend é o *script* de inicio:

```
cd backend
./start.sh
```

Alternativamente, pódense levantar só os contedores:

```
cd backend
podman compose up -d # engadir --profile gpu se hai GPU dispoñible
```

Portos por defecto: PostgreSQL 5432, MinIO 9000/9001 (API/consola), Ollama 11434.

A.4. Execución do backend

```
cd backend
./gradlew bootRun
```

O servidor escoita no porto 8082. A configuración (base de datos, *keystore* de sinatura JWT, MinIO e tarefas programadas) atópase en `src/main/resources/application.yaml`. A documentación interactiva da API queda dispoñible en `/swagger-ui.html`.

A.5. Execución do frontend

```
cd frontend
npm install
npm start # inicia o servidor de desenvolvemento de Expo
```

A dirección do backend configúrase na constante `BASE_URL` de `frontend/constants/constants.tsx`, que debe apuntar ao enderezo accesible do servidor desde o dispositivo. A aplicación pode abrirse en Expo Go ou nun emulador (`npm run android` / `npm run ios`).

A.6. Contrato OpenAPI e xeración de código

A API especificase en `backend/src/main/resources/openapi/`, cun documento principal (`api.yaml`) que referencia ficheiros de rutas e esquemas. A partir del xérase código:

```
# Backend: interfaces de controladores e DTO
cd backend && ./gradlew openApiGenerate

# Frontend: tipos TypeScript
cd frontend && npm run generate:types
```

Ao editar ficheiros referenciados con `$ref` (rutas ou esquemas), pode ser preciso forzar a rexeneración, xa que a tarefa de Gradle só observa cambios en `api.yaml`.

A.7. Execución das probas

As probas de integración (E2E) empregan Testcontainers, que require un *socket* compatible con Docker. Con Podman *rootless*, débese indicar a súa ruta:

```
cd backend
export DOCKER_HOST=unix:///run/user/$(id -u)/podman/podman.sock
export TESTCONTAINERS_RYUK_DISABLED=true
./gradlew test
```

Ao rematar, o informe de cobertura JaCoCo xérase en `backend/build/reports/jacoco/test/html/index.html`. A análise estática SpotBugs está dispoñible mediante a tarefa correspondente de Gradle.

A.8. Configuración e segredos

A configuración do backend (credenciais da base de datos, contrasinal do *keystore* JWT, claves de acceso a MinIO e parámetros das tarefas programadas) reside en `application.yaml`. Para un despregamento en produción recoméndase externalizar estes valores mediante variables de entorno e non incluílos no control de versións.

A.9. Integración continua

O repositorio inclúe fluxos de traballo de GitHub Actions que executan a construción e as probas do backend, así como unha análise de seguridade do código (CodeQL).

B. Manual de usuario

Este manual é autocontido e está dirixido ás persoas que utilicen Telaria. Describe a instalación e o uso de todas as funcionalidades. As capturas de pantalla insírense nos espazos reservados ao efecto.

B.1. Instalación e acceso

Telaria é unha aplicación móbil para iOS e Android. Durante o desenvolvemento pode executarse a través de Expo Go ou instalando unha compilación da aplicación. Ao abrila por primeira vez, o usuario debe rexistrarse ou iniciar sesión.

B.2. Rexistro e inicio de sesión

Na pantalla inicial pódese crear unha conta nova indicando nome de usuario, correo electrónico e contrasinal (con confirmación), ou iniciar sesión cunha conta existente. A sesión mantense aberta entre usos.

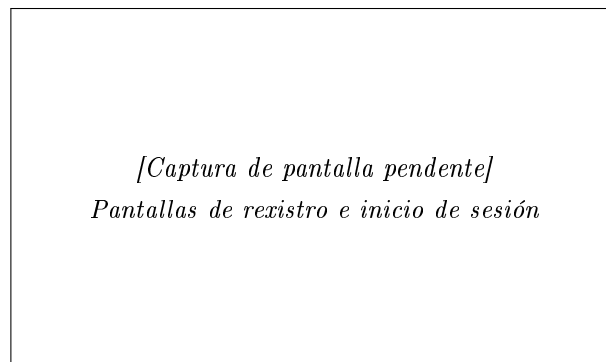


Figura 12: Pantallas de rexistro e inicio de sesión

B.3. Pantalla principal: as miñas viaxes

Tras iniciar sesión, móstrase a lista de viaxes do usuario. Desde aquí pódese acceder a unha viaxe, crear unha nova ou unirse a unha existente, así como abrir o menú lateral para acceder a amigos, invitacións, perfil e axustes.

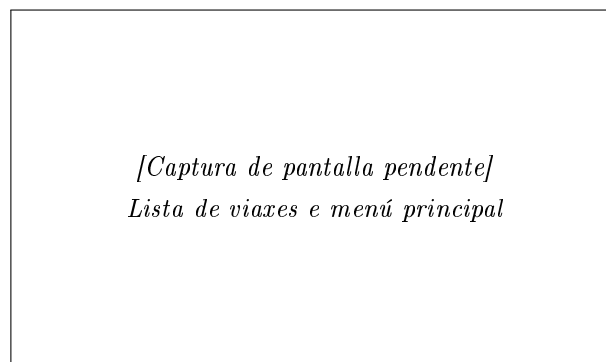
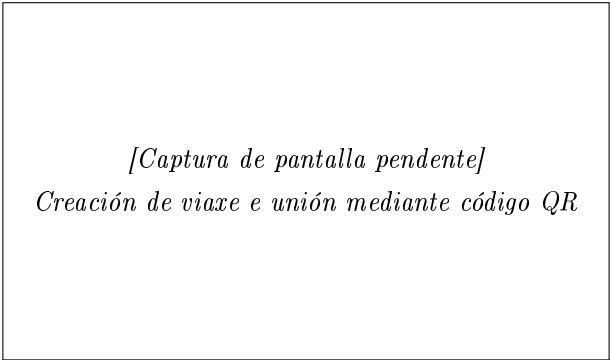


Figura 13: Lista de viaxes e menú principal

B.4. Crear e unirse a viaxes

Para crear unha viaxe abonda con indicar o seu nome; opcionalmente pódense convidar amigos no momento da creación. Para unirse a unha viaxe existente pódese empregar unha invitación recibida, enviar unha solicitude de unión ou escanear un código QR.

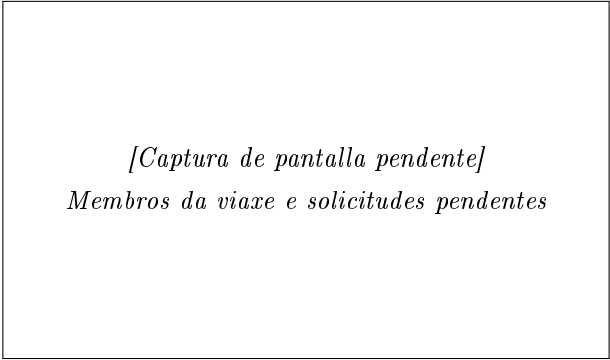


[Captura de pantalla pendiente]
Creación de viaxe e unión mediante código QR

Figura 14: Creación de viaxe e unión mediante código QR

B.5. Xestión de membros

Dentro dunha viaxe, calquera membro pode convidar outros usuarios e xestionar as solicitudes de unión pendentes. Non existen roles: todos os membros teñen as mesmas capacidades.

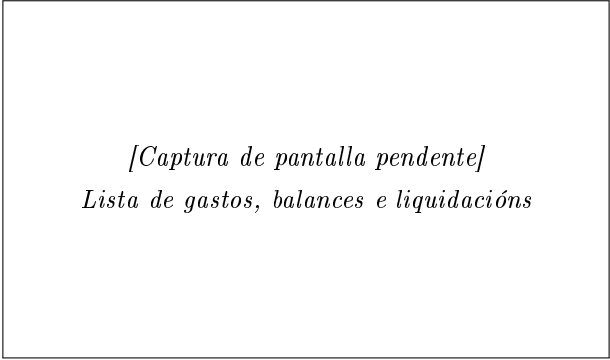


[Captura de pantalla pendiente]
Membros da viaxe e solicitudes pendentes

Figura 15: Membros da viaxe e solicitudes pendentes

B.6. Gastos e liquidacións

No apartado de gastos rexístranse os pagamentos indicando importe, concepto, categoría, quen pagou e entre quen se reparte. A aplicación calcula automaticamente os balances (quen debe a quen) e propón as liquidacións que minimizan o número de transferencias. As débedas saldadas rexístranse no histórico de liquidacións.



[Captura de pantalla pendiente]
Lista de gastos, balances e liquidacións

Figura 16: Lista de gastos, balances e liquidacións

B.7. Eventos e calendario

O calendario da viaxe permite crear eventos con nome, data e hora de inicio, duración e unha localización opcional. Os eventos amósanse organizados por data.

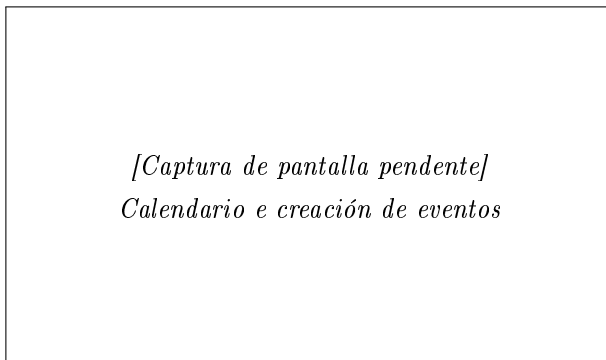


Figura 17: Calendario e creación de eventos

B.8. Documentos compartidos

Os membros poden subir documentos relevantes para a viaxe (reservas, billetes, etc.) e consúltalos ou elimínalos. Os documentos de imaxe amosan unha miniatura de previsualización.

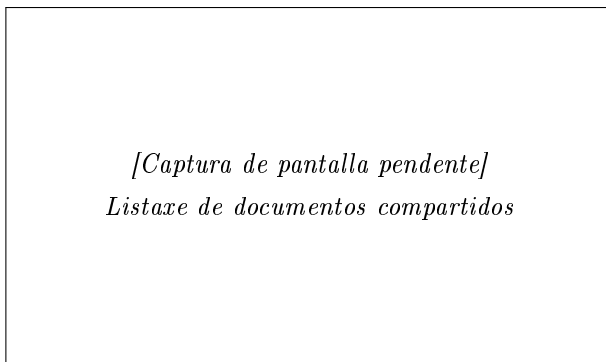


Figura 18: Listaxe de documentos compartidos

B.9. Chat de grupo

Cada viaxe dispón dun chat común no que os membros intercambian mensaxes en tempo real.

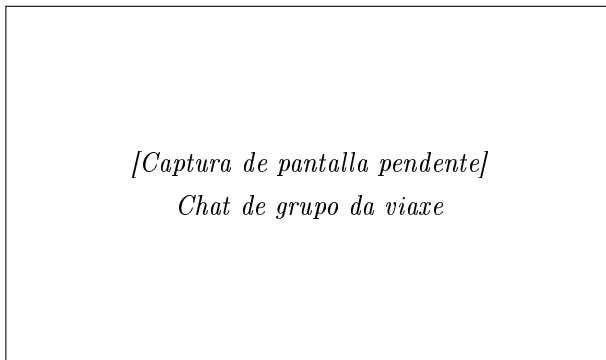


Figura 19: Chat de grupo da viaxe

B.10. Asistente de intelixencia artificial

Cada viaxe inclúe un asistente conversacional que responde preguntas tendo en conta o contexto da viaxe (eventos e gastos). A conversa garda un historial persistente.

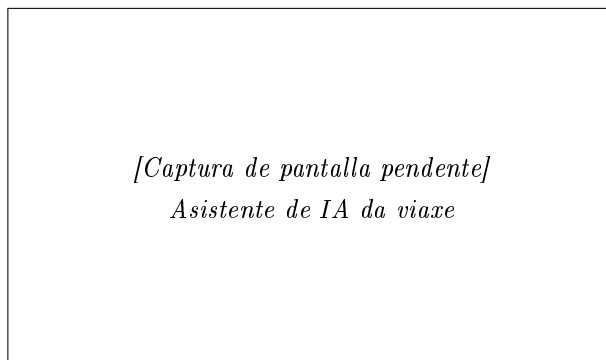


Figura 20: Asistente de IA da viaxe

B.11. Amizades

A sección de amigos permite enviar e xestionar solicitudes de amizade (por identificador ou código QR), consultar a lista de amigos e eliminar amizades. Os amigos poden convidarse ás viaxes de forma máis áxil.

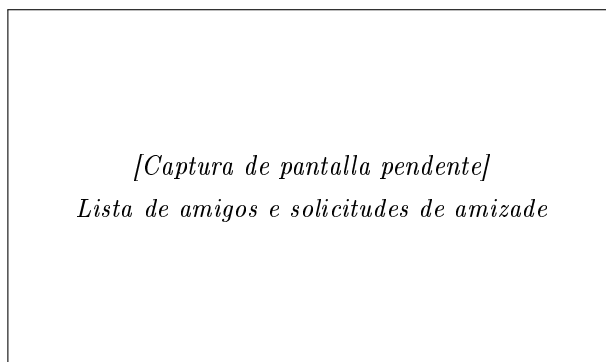


Figura 21: Lista de amigos e solicitudes de amizade

B.12. Perfil e axustes

No perfil pódese consultar e actualizar o nome de usuario e o correo electrónico (o cambio de correo require o contrasinal), cambiar o contrasinal e establecer un avatar. Nos axustes configúrase o idioma (galego, castelán ou inglés), o tema (claro ou escuro) e o aforro de datos.

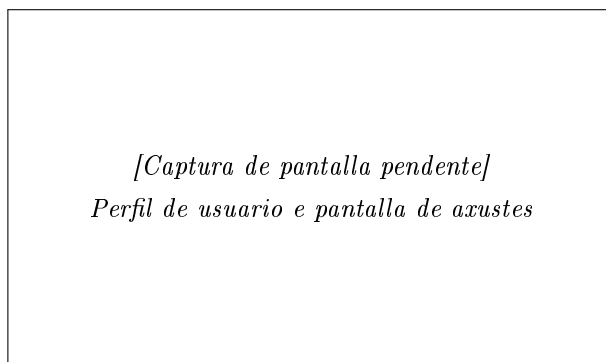


Figura 22: Perfil de usuario e pantalla de axustes

B.13. Eliminación da conta

Desde os axustes, o usuario pode eliminar a súa conta. A operación require a confirmación do contrasinal e elimina os datos persoais asociados.

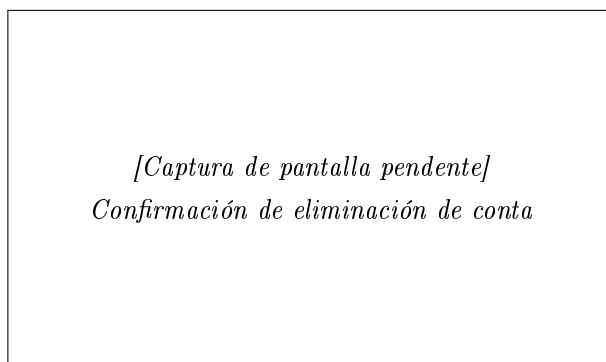


Figura 23: Confirmación de eliminación de conta

B.14. Mensaxes de erro comúns

- **Credenciais incorrectas:** o correo ou o contrasinal non son válidos no inicio de sesión.
- **Correo xa rexistrado:** existe xa unha conta con ese correo ao rexistrarse.
- **Acción xa realizada:** por exemplo, o usuario xa é membro da viaxe ou xa foi convidado.
- **Sen conexión:** a aplicación require conexión á rede; comprobe a conectividade e que o servidor estea accesible.